

VITyarthi - Build Your Own Project

Flipped Course Evaluation Project Report

CAMPUSHUB

Academic Resource & Peer Study Group Scheduling System

Course Domain: Software Engineering & Python Application Development

Student Name: Tanmay Chaubey

Academic Term: Fall 2026

Architecture: Pure Python 3 Standard Architecture & Relational SQLite

Repository: Complete Source Code, Unit Tests & Documentation Included

2. Introduction

Peer tutoring and self-organized study sessions are cornerstones of collegiate academic success. When classmates collaborate on algorithmic puzzles, systems programming labs, or database designs, their understanding and course grades improve dramatically.

However, running study circles over WhatsApp or Discord is notoriously frustrating. Meeting timings get lost in conversational clutter, peer mentors double-book review slots, and shared study materials get buried or expire. **CampusHub** is an academic platform written in pure Python that brings order to student collaboration through

course-indexed study groups, conflict-free scheduling, categorized resource indexing, attendance analytics, and an integrated academic integrity engine.

3. Problem Statement

College students face clear, recurring obstacles when attempting to organize study groups:

- **Cluttered Communication:** Meeting details and locations disappear under hundreds of chat messages.
- **Double-Booking:** Tutors handling multiple subjects frequently schedule conflicting study slots on the same day.
- **Unindexed Study Materials:** Notes, past paper solutions, and code files are scattered across personal cloud links that get lost.
- **Zero Accountability:** No record exists of who attended review meets or whether the tutor's explanations were clear.
- **Academic Integrity Risks:** Collaborative groups need tools to ensure shared materials are original rather than plagiarized or synthetic code.

4. Functional Requirements

Module 1: User & Study Group Management

- **User Authentication:** Secure student and tutor sign-up with email validation, password checks, and salted PBKDF2 SHA-256 hashing.
- **Course Study Groups:** Creation of subject cohorts (e.g. CSE2001) with capacity controls.
- **Approval Workflows:** Direct enrollment for public groups and leader authorization queues for private groups.

Module 2: Session Scheduler & Conflict Detector

- **Conflict-Free Booking:** Review session scheduling with room locations or virtual video links.
- **Algorithmic Interval Collision Check:** Mathematical detection preventing overlapping bookings: $(StartA < EndB) \text{ AND } (EndA > StartB)$.
- **RSVP & Verified Attendance:** Seat reservation and timestamped check-in attendance verification.

Academic Resource Repository Submodule

- **Categorization:** Notes, Past Papers, Code Scripts, Slides, and Reference Guides.
- **Tag Indexing & Multi-Search:** Fast filtering by keyword, course code, and comma-separated tags.
- **Download Frequency Auditing:** Dynamic counter tracking material popularity.

Module 3: Peer Review & Analytics Engine

- **5-Star Review System:** Attendees rate sessions and provide written feedback; hosts are prevented from self-rating.
- **Turnout Ratios:** Automated computation of attendance percentage: $(\text{Verified} / \text{Total RSVPs}) * 100\%$.
- **Performance Dossiers:** Auto-generated academic performance reports for cohorts.

Academic Integrity & AI Code Detection Module

- **Plagiarism Comparison Engine:** Token winnowing algorithm to compute Jaccard similarity and containment percentages.

- **AI Code Detector:** Calculates Shannon token entropy, Type-Token Ratio, line length burstiness variance, and checks for AI boilerplate.

5. Non-Functional Requirements

- **Performance:** Sub-50ms query latency via database indexes on session times, courses, and usernames. SQLite Write-Ahead Logging (WAL) enabled.
- **Security:** Salted PBKDF2 SHA-256 password hashing (100,000 rounds) and parameterized SQL queries to prevent SQL injection.
- **Zero-Dependency Execution:** Runs natively on Python 3 standard library with zero external pip package installation overhead.
- **Reliability:** ACID transactional integrity with automated rollback upon operational exceptions.
- **Maintainability:** Clean layered Service-Model-Repository architecture across 13 distinct Python modules.

6. System Architecture



7. Design Diagrams

7.1 Entity-Relationship (ER) Schema

Table Name	Primary Key	Foreign Keys	Key Fields
users	id	None	username, email, password_hash, salt, role, department
study_groups	id	creator_id (users)	name, course_code, max_members, is_private
group_members	id	group_id, user_id	role_in_group, status (ACTIVE/PENDING)
study_sessions	id	group_id, host_id	title, start_time, end_time, status, location_or_link
session_attendance	id	session_id, user_id	status (RSVP/ATTENDED), check_in_time
study_resources	id	group_id, uploader_id	title, resource_type, file_or_url, tags, download_count
session_reviews	id	session_id, reviewer_id	rating (1-5), comments, reviewee_id

8. Design Decisions & Rationale

- **Standard Library Focus:** Eliminates installation barriers so evaluators can run the project on any standard Python setup.
- **Interval Collision Logic:** Prevents scheduling clashes algorithmically at insertion time, providing 100% schedule reliability.

- **MOSS-Style Winnowing for Plagiarism:** Fingerprints token streams so comparisons remain robust against superficial variable renaming.

9. Implementation Details

The codebase is organized into modular Python files:

- `campushub/models/`: Defines data transfer classes (`User`, `StudyGroup`, `StudySession`, `StudyResource`, `SessionReview`).
- `campushub/services/`: Implements business logic (`AuthService`, `GroupService`, `SchedulerService`, `ResourceService`, `AnalyticsService`, `IntegrityService`).
- `campushub/database/`: Provides connection handling, schema definition, and ACID transactions.
- `campushub/cli/`: Formats clean terminal menus and tables.
- `campushub/web/`: Provides zero-dependency HTTP server with JSON REST APIs and frontend dashboard.

10. Screenshots & Results

Sample Analytics Report:

```
=====
                        CAMPUSHUB ACADEMIC STUDY GROUP ANALYTICS REPORT
=====
Group Name       : Algorithms & DS Mastery
Course Code      : CSE2001
Active Members   : 3
-----
SESSION PERFORMANCE METRICS:
* Total Sessions Created : 1
* Completed Sessions     : 1
* Currently Scheduled    : 0
* Cancelled Sessions     : 0
-----
ATTENDANCE & ENGAGEMENT:
* Total Member RSVPs     : 2
* Verified Attendances   : 2
* Attendance Rate        : 100.0%
-----
RESOURCE SHARING & COLLABORATION:
* Total Materials Uploaded : 1
* Material Access Count    : 2
-----
QUALITY & PEER EVALUATION:
* Average Satisfaction    : 5.0 / 5.0
* Reviews Evaluated       : 1
=====
```

11. Testing Approach

Automated unit tests were built using Python's standard unittest framework with in-memory SQLite isolation. All 23 test cases pass successfully:

Test Name	Module	Purpose	Status
test_successful_registration_and_login	AuthService	Validate password hashing & login flow	PASS
test_duplicate_username_rejection	AuthService	Reject duplicate username registration	PASS
test_weak_password_validation	AuthService	Enforce alphanumeric password length	PASS
test_create_group_and_leader_assignment	GroupService	Auto-assign creator as Leader	PASS
test_join_private_group_and_approval	GroupService	Leader approval workflow verification	PASS
test_host_conflict_detection_algorithm	SchedulerService	Reject overlapping bookings for tutor	PASS
test_group_conflict_detection	SchedulerService	Reject simultaneous sessions for group	PASS
test_rsvp_and_attendance_checkin	SchedulerService	Verify seat reservation & check-in	PASS
test_self_review_prohibition	AnalyticsService	Prevent hosts from rating themselves	PASS

Test Name	Module	Purpose	Status
test_group_analytics_metrics	AnalyticsService	Validate attendance ratio math	PASS
test_identical_plagiarism_detection	IntegrityService	Detect matching token fingerprints in code	PASS
test_detect_ai_code_markers	IntegrityService	Score synthetic AI code heuristics	PASS

12. Challenges Faced

- **Interval Collision Edge Cases:** Solved by using universal overlap logic: $(StartA < EndB) \text{ AND } (EndA > StartB)$.
- **In-Memory Database Persistence during Tests:** Retained persistent connection pointers during unit test executions.
- **Zero-Dependency Browser GUI:** Implemented using native `http.server` and Vanilla JS.

13. Learnings & Key Takeaways

Demonstrated the capability of building production-grade, maintainable software systems using pure Python principles, strict separation of concerns, and robust test-driven development.

14. Future Enhancements

- Two-way integration with Google Calendar and Outlook APIs.
- Automated SMS and email reminders prior to study meets.

- Machine learning recommendations matching study cohorts based on common course schedules.

15. References

1. Python Software Foundation. *Python 3 Standard Library Documentation*.
<https://docs.python.org/3/>
2. Schleimer, S., Wilkerson, D. S., & Aiken, A. *Winnowing: Local Algorithms for Document Fingerprinting*. ACM SIGMOD, 2003.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill, 2019.
4. Martin, R. C. *Clean Architecture: A Craftsman's Guide to Software Structure*. Prentice Hall, 2017.
5. OWASP Foundation. *Password Storage Cheat Sheet*.
<https://cheatsheetseries.owasp.org/>